



Scalable Model Checking for Railway Interlocking Systems

**Multi-Process Implementation
in Python**

Jacopo Bruscaaglioni

Introduction

Railway Interlocking

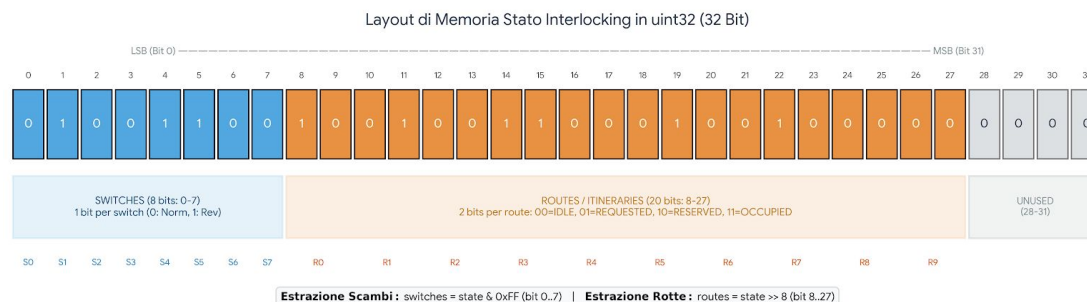
- **Domain:** Computer-Based Interlocking (CBI) in safety-critical railway signaling.
- **Goal:** Exhaustive verification of critical safety invariants (collision freedom, derailment prevention, route conflict isolation) for a system of 2 track, 4 Switches per track and 5 Routes per track..
- **The Computational Bottleneck:** *The State Space Explosion Problem*. Adding tracks, switches, and routes causes the reachability graph to scale combinatorially, making sequential exhaustive verification intractable in reasonable time.

32-Bit State Representation

Bit Allocation for State S:

- ❑ **N_{sw}** : number of switches
- ❑ **N_{rt}** : number of routes
 - Bits 0 ... N_{sw} - 1: Switch alignments (0: Normal, 1: Reverse).
 - Bits N_{sw} ... N_{sw} + 2N_{rt} - 1: 2-bit route status (00: IDLE, 01: REQUESTED, 10: RESERVED, 11: OCCUPIED).

Bitwise Arithmetic: Rapid transition checking and topological safety validation via single-cycle bitwise masks (**AND**, **XOR**, bit shifts).



Introduction

Safety Invariants

- **Invariant 1: Rival Routes (*Conflicting Paths*):**
 - Evaluates topologically incompatible routes sharing identical track segments or switches.
 - *Hazard Condition*: Two routes that share at least one **Switch** are locked or active simultaneously (state $\geq 2 \rightarrow$ *RESERVED* or *OCCUPIED*) .
- **Invariant 2: Switch Alignment Consistency:**
 - Ensures that each commanded Route has its corresponding Switches **properly positioned**.
 - *Hazard Condition*: A route is *locked*, but one or more underlying Switches are **misaligned**.
- **Computational Complexity:** The workload is strictly *embarrassingly parallel*; each state is verified autonomously with zero cross-state dependencies.

– *inv. 1* –

```
for p in range(num_pairs):
    r1 = (routes >> pairs_arr[p, 0]) & 3
    r2 = (routes >> pairs_arr[p, 1]) & 3
    if r1 ≥ 2 and r2 ≥ 2:
        is_unsafe = 1
        break
```

– *inv. 2* –

```
if is_unsafe == 0:
    for r_idx in range(n_rt):
        r_st = (routes >> (r_idx * 2)) & 3
        if r_st == 2 and
           ((switches ^ (r_idx * 31))
            & 170 == 0):
            is_unsafe = 1
            break
```

Proposed Approach

Computational Architectures

Three distinct evaluation backends:

- 1) **Sequential CPU:** Single-threaded baseline implementation in Python.
- 2) **Multi-Process CPU + Shared Memory:** Native multi-core parallelization using Python's **multiprocessing** and **shared memory**.
- 3) **NVIDIA CUDA (SIMT)** : Parallel 1D grid distribution on the streaming multiprocessors (SMs), executing the *Safety Kernel* directly on Device memory.

The Python Parallelism Challenge (GIL):

- In CPython, the Global Interpreter Lock (GIL) *limits execution to a single OS thread at any time*.
- Standard *multi-threading* cannot leverage multicore CPUs for **CPU-bound loops**.
- True CPU parallel scaling requires spawning isolated OS processes (*multiprocessing.Pool*).
- **POSIX Shared Memory** is employed so that *multiple processes* can access large data arrays, maximizing per-core performance while bypassing the Global Interpreter Lock (GIL)

CUDA Acceleration via Numba JIT:

- **Native GPU Execution:** Uses `@cuda.jit` to compile Python logic directly into PTX (*Parallel Thread Execution*) machine code via LLVM (*Low Level Virtual Machine*), bypassing CPython and the GIL completely.
- **Massive Thread-Level Parallelism:** Maps each 32-bit state directly to a distinct GPU thread (`cuda.grid(1)`) across thousands of CUDA cores.
- **Device Memory Locality:** Eliminates IPC overhead by executing bitwise invariant evaluations directly inside high-bandwidth VRAM.

Experimental Setup & Methodology

Hardware Environment (Kaggle Cloud Platform):

- **Host CPU:** Intel Xeon Processor @ 2.20GHz (2 physical cores / 4 vCPUs via Hyper-Threading, 38.5 MB L3 Cache, 30 GB System RAM).
- **Target GPU:** NVIDIA Tesla T4 (Turing Architecture, 40 Streaming Multiprocessors, 2,560 CUDA Cores, 16 GB GDDR6 VRAM, PCIe Gen3).

Software Toolchain & Environment:

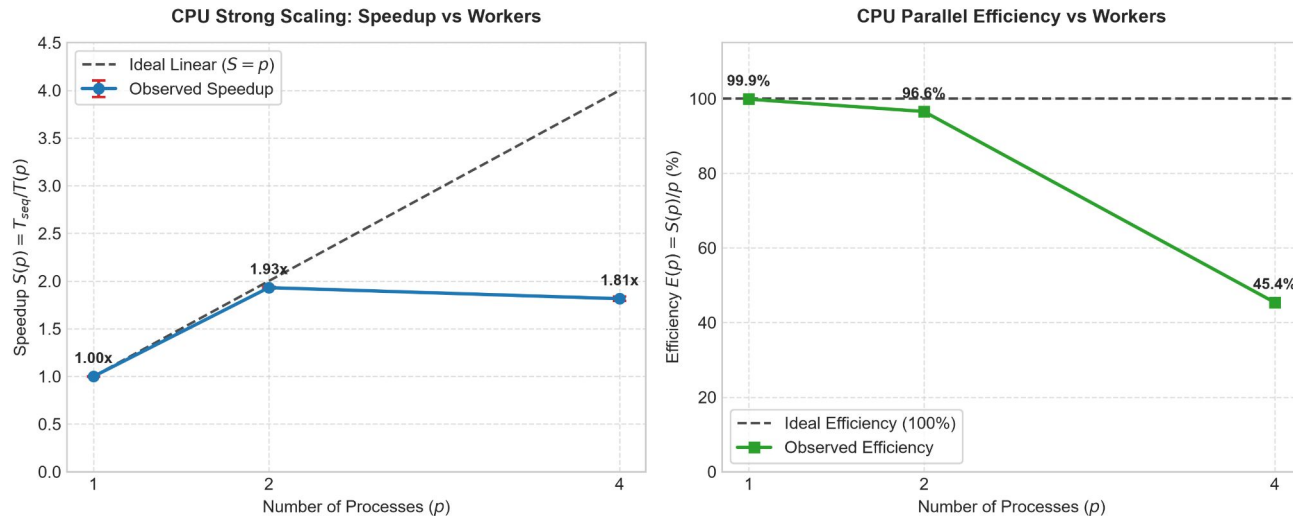
- **Platform & OS:** Kaggle Notebook Container on Ubuntu 22.04 LTS (Linux kernel 5.15+).
- **Runtime Stack:** Python 3.10+, NumPy, Numba (LLVM JIT Compiler), NVIDIA CUDA Toolkit 12.x, NVIDIA Driver 535+.

Measurement Protocol:

- **Warm-up:** 2 preliminary warm-up runs discarded to eliminate cold-start cache latency and JIT compilation overhead.
- **Statistical Rigor:** 5 timed iterations recorded per configuration using `time.perf_counter()` (reporting **mean**, **standard deviation**, **min**, and **max**).
- **Strict Timing Scope:** Disk I/O, process initialization, and Host-to-Device memory allocations are strictly **excluded** from pure computation timings.
- **Automated Correctness:** Bit-exact verification against the sequential baseline via `numpy.array_equal()`.



Multicore CPU Scaling Analysis

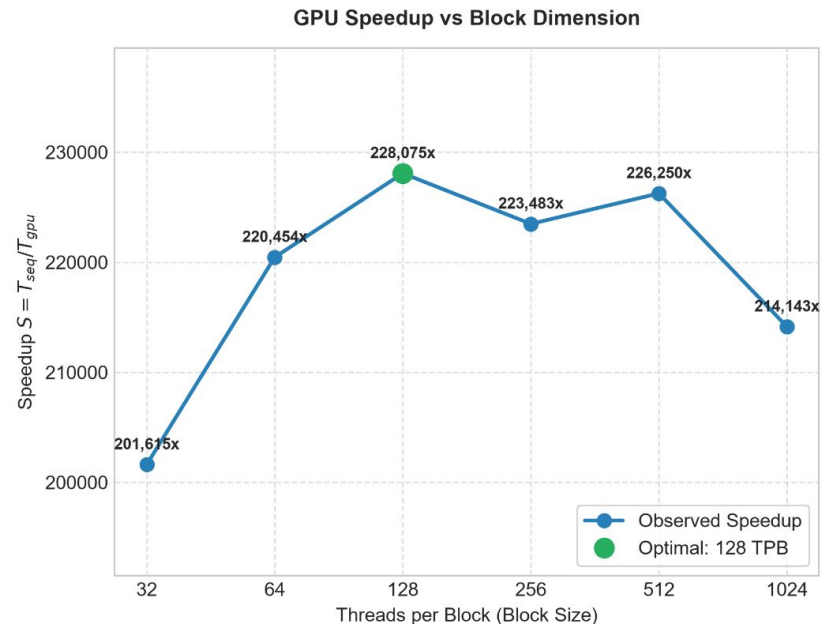
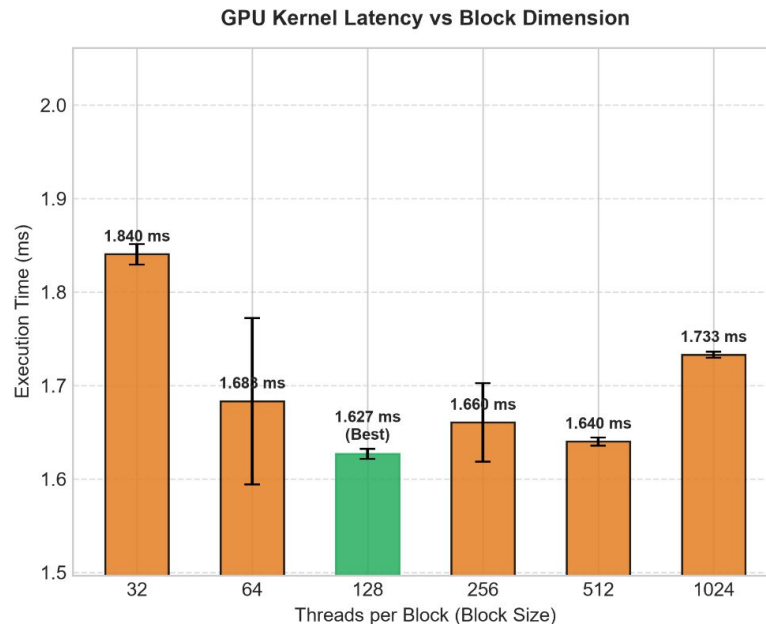


Near-Linear Scaling ($p \leq 2$): Achieves $1.93\times$ speedup with 96.6% efficiency on 2 physical cores, validating the zero-copy shared memory approach.

Oversubscription Penalty ($p = 4$): Speedup drops to $1.81\times$ (45.4% efficiency) as 4 worker processes compete for 2 physical execution cores, forcing **context switching**.

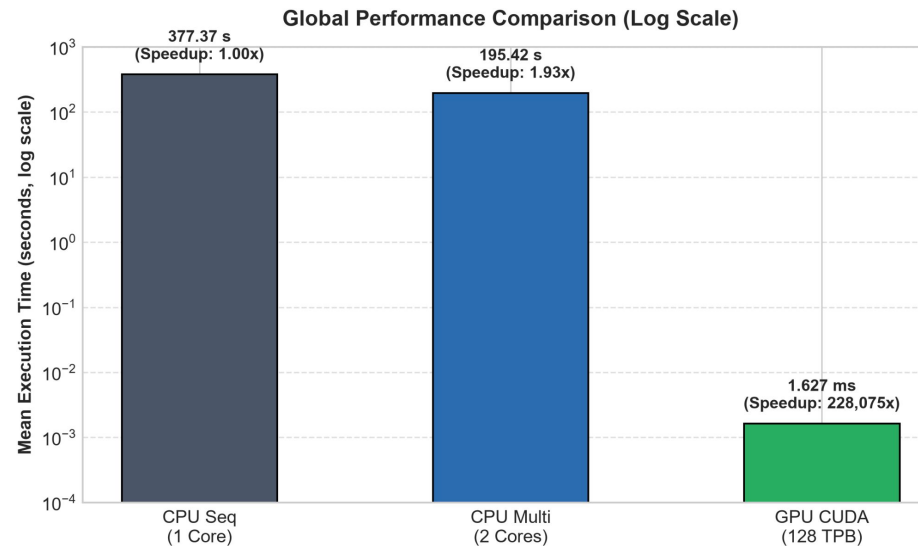
Hardware Limit: Confirms that performance is strictly bounded by physical CPU core availability rather than algorithmic serialization.

CUDA GPU Performance & Block Tuning



- **Optimal Configuration (128 TPB):** Delivers the lowest kernel latency (1.627 ms) and maximum speedup (228,075×) by maximizing active warp occupancy on the *Tesla T4 Streaming Multiprocessors*.
- **Under-Occupancy at Low Block Sizes (32–64 TPB):** Slower execution (1.840 ms at 32 TPB) due to *insufficient warps* available to hide arithmetic latency.
- **Resource Saturation at 1024 TPB:** Latency regresses to 1.733 ms (214,143×) as maximum thread limits per SM constrain parallel block scheduling flexibility.
- **Consistent Multi-Order Speedup:** All block sizes sustain a massive acceleration factor exceeding 200,000×, confirming the efficiency of 32-bit coalesced memory reads across the GPU.

Global Performance Comparison



Master Summary Table:

- Sequential baseline vs. Best Multicore configuration vs. CUDA GPU kernel.
- Absolute execution times, overall speedups, and throughput (states verified per second).

CUDA provides **massive order-of-magnitude speedups** for state checking, while multicore CPU provides an accessible mid-tier speedup without requiring specialized GPU hardware.

Results Table: Multiprocessing

Workers	Mean Time (s)	Std Dev (s)	Min Time (s)	Max Time (s)	Speedup	Efficiency	Validation
1	377.86	0.85	376.89	378.93	1.00×	99.9%	PASSED
2	195.42	0.70	194.58	196.36	1.93×	96.6%	PASSED
4	207.93	2.84	204.43	211.62	1.81×	45.4%	PASSED



Results Table: CUDA GPU (Tesla T4)

Threads/Block	Mean Time (ms)	Std Dev (ms)	Min Time (ms)	Max Time (ms)	Speedup	Efficiency	Validation
32	1.840	0.011	1.825	1.855	201,615×	N/A	PASSED
64	1.683	0.089	1.628	1.860	220,454×	N/A	PASSED
128	1.627	0.005	1.620	1.636	228,075×	N/A	PASSED
256	1.660	0.042	1.624	1.734	223,483×	N/A	PASSED
512	1.640	0.004	1.637	1.648	226,250×	N/A	PASSED
1024	1.733	0.003	1.727	1.737	214,143×	N/A	PASSED



Conclusions

- **Global Performance Comparison:** The CUDA backend abundantly outperforms all CPU approaches, reducing execution latency from **377.37s to 1.63ms** (a **228,075x** speedup).
- **Complexity Trade-off:** CUDA achieves peak hardware throughput at the cost of managing explicit VRAM memory allocations and 1D execution grid tuning.
- **Accessible CPU Scaling:** Python Multiprocessing with POSIX shared memory provides an immediate, noticeable speedup (**1.93x** at **96.6%** efficiency) without requiring dedicated GPU accelerators.
- **Bit-Exact Integrity:** All parallel implementations achieve **100% validation consistency** against the *ground-truth sequential baseline* (**PASSED**)

